

Politechnika Gdańska

Katedra Optoelektroniki i Systemów Elektronicznych

Organizacja i Oprogramowanie Systemów Elektronicznych

Michał Kowalewski

Wykorzystanie środowiska MATLAB w programowaniu systemów elektronicznych



Politechnika Gdańska
2011

1. Wprowadzenie

Celem ćwiczenia jest oraz przedstawienie możliwości konstruowania graficznego interfejsu użytkownika w środowisku MATLAB oraz wykorzystanie funkcji biblioteki *Instrument Control Toolbox* do sterowania pracą oscyloskopu Rigol DS1052E.

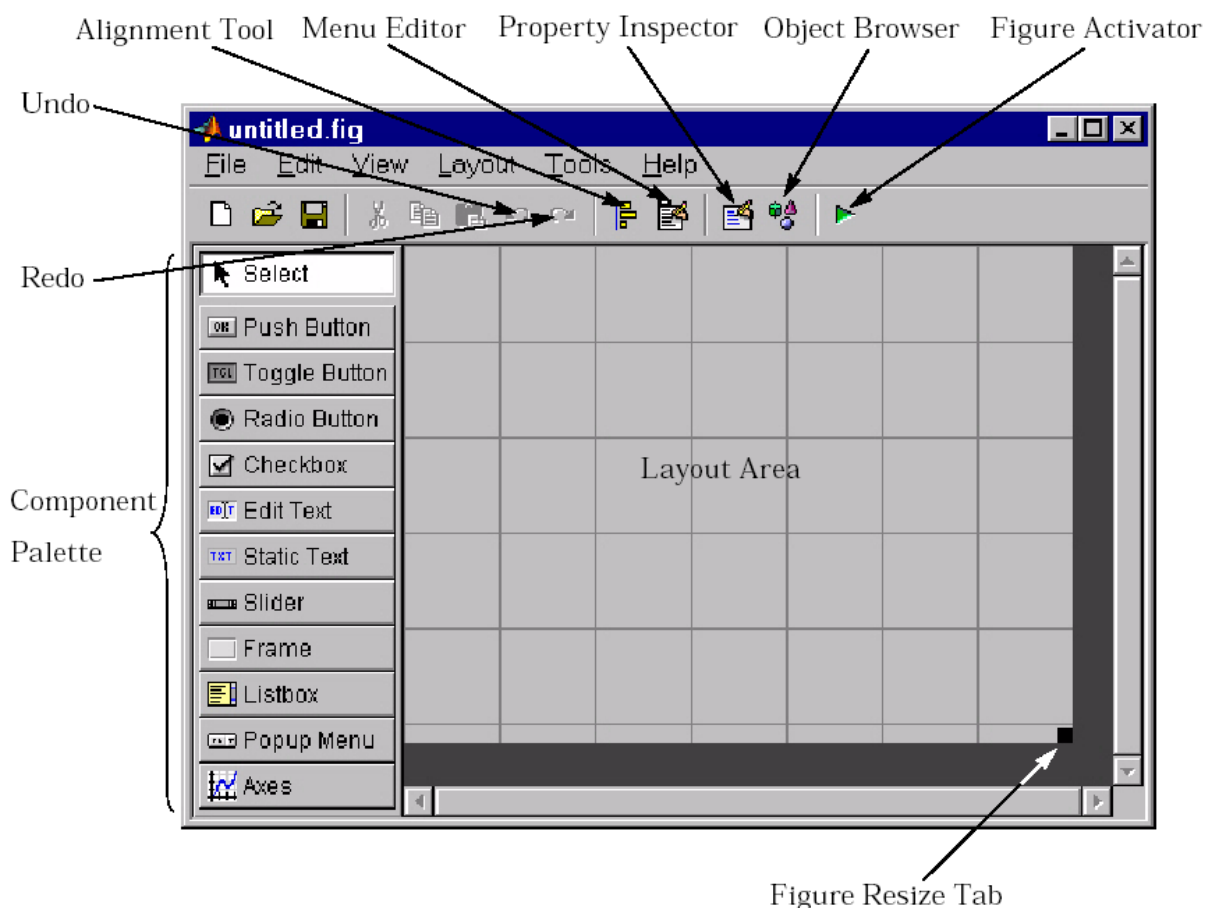
2. Projektowanie raficznego interfejsu użytkownika w środowisku MATLAB

W srodowisku MATLAB do projektowania graficznego interfejsu użytkownika przeznaczony jest pakiet narzędzi GUIDE (*Graphical User Interface Development Environment*), uruchamiany poleceniem *guide*.

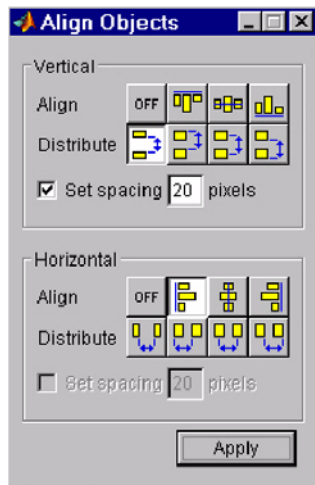
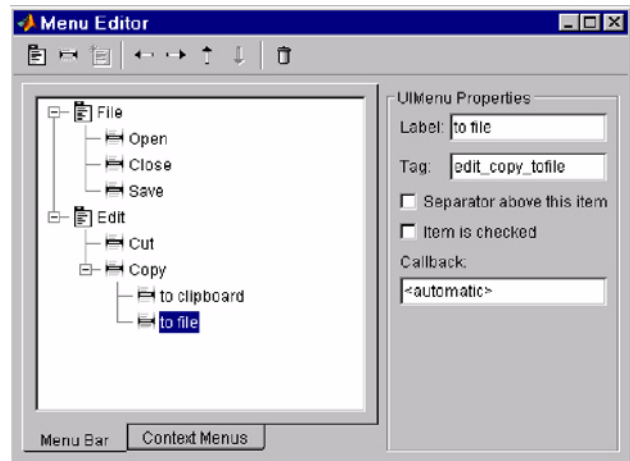
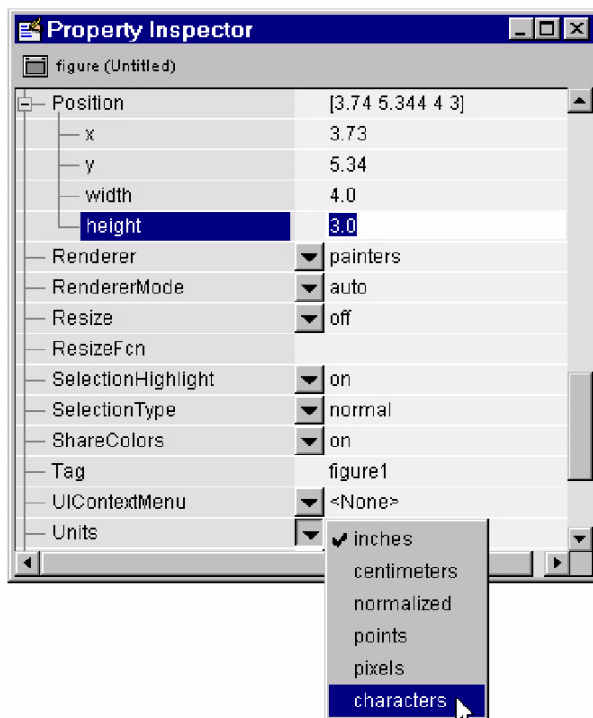
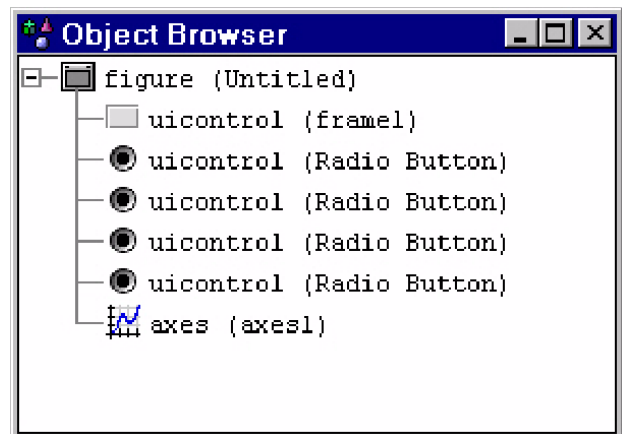
Głównym składnikiem pakietu GUIDE jest edytor interfejsu użytkownika *Layout Editor*, pokazany na rys. 1. GUIDE zawiera kilka narzędzi wspomagających projektowanie GUI:

- **Alignment Tool** – rozmieszczanie komponentów na panelu (Rys. 2),
- **Property Inspector** – edycja właściwości komponentów (Rys. 3),
- **Object Browser** – podgląd listy komponentów na panelu (Rys. 4),
- **Menu Editor** – edytor menu na panelu oraz menu kontekstowego (Rys. 5).

Narzędzia te uruchamia się za pomocą przycisków na pasku narzędzi edytora interfejsu użytkownika (Rys. 1).



Rys. 1. Edytor interfejsu użytkownika *Layout Editor*

Rys. 2. Okienko edycyjne rozmieszczenia komponentów (*Alignment Tool*)Rys. 4. Okienko edycyjne menu (*Menu Editor*)Rys. 3. Okienko edycyjne właściwości (*Property Inspector*)Rys. 5. Okienko podglądu listy komponentów na panelu (*Object Browser*)

2. Przyrządy wykorzystywane w ćwiczeniu

- komputer z zainstalowanym oprogramowaniem:
 - o MATLAB R12 z biblioteką *Instrument Control Toolbox*
 - o Agilent IO Control
- oscyloskop Rigol DS1052E
- generator funkcyjny

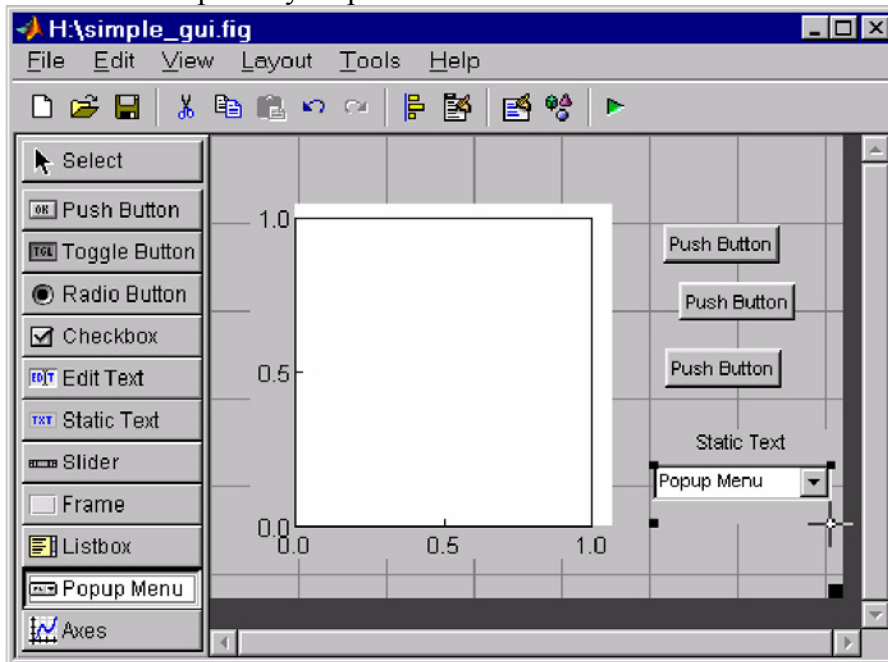
3. Zadania laboratoryjne

3.1. Przygotowanie do ćwiczenia

- Sprawdzić połączenie oscyloskopu z komputerem za pomocą przewodu RS232

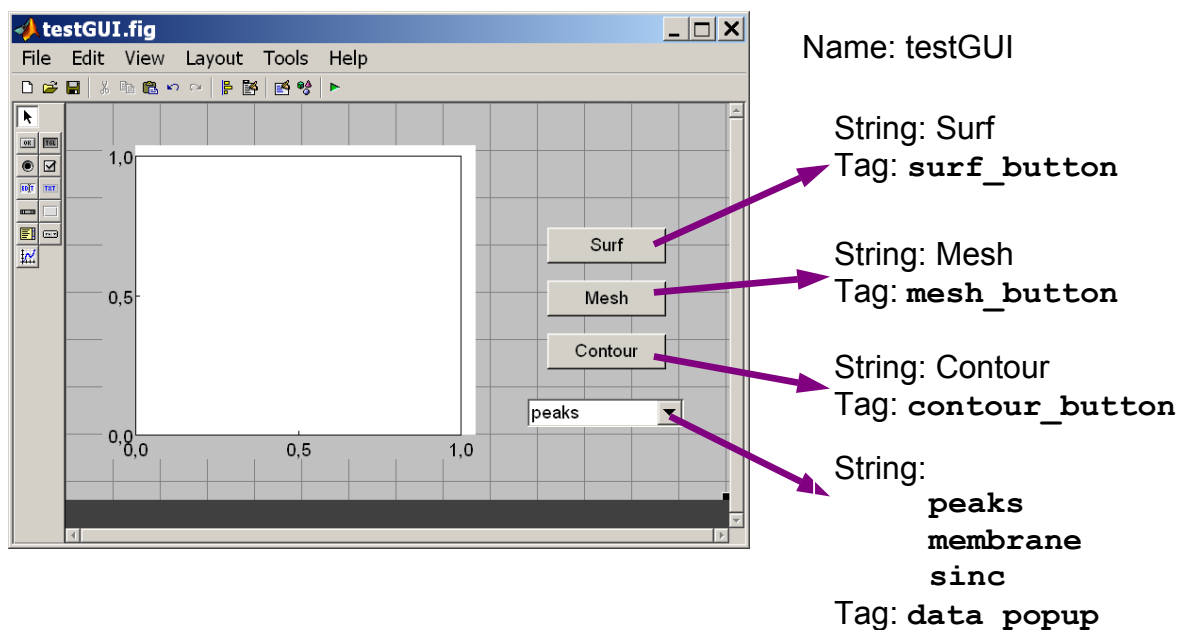
3.2. Projektowanie interfejsu graficznego w środowisku MATLAB

- Uruchomić MATLABa oraz edytor GUI poleceniem **guide**.
- Rozmieścić komponenty na panelu.

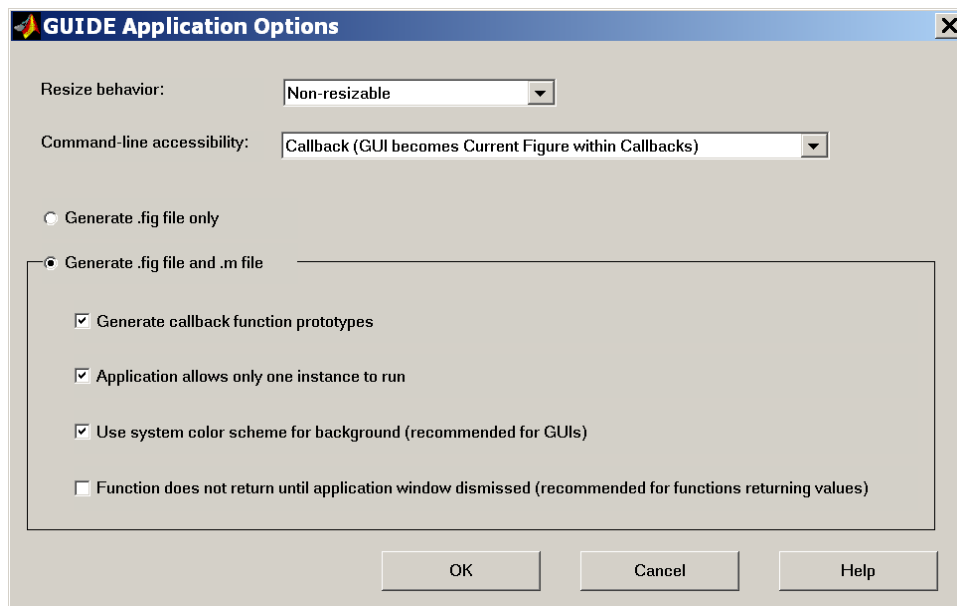


- Zapoznać się z narzędziami wspomagającymi projektowanie GUI.

- Przeprowadzić edycję właściwości kontrolki na panelu (narzędzie Property Inspector)



- Skonfigurować kreator aplikacji zgodnie z poniższym rysunkiem.



f) W celu utworzenia szkieletu programu dokonać aktywacji panelu (Tools/Activate Figure lub File/Save As...).
Projekt zapisać pod nazwą **testGUI**. Powinny zostać utworzone dwa pliki: **testGUI.fig** i **testGUI.m**.

g) Uzupełnić kod źródłowy programu korzystając z wbudowanego edytora tekstowego (polecenie **edit testGUI**).

```
function varargout = testGUI(varargin)
    if nargin == 0 % If no input arguments, launch the GUI
    % ...
    handles = guihandles(fig);
    guidata(fig, handles);
% początek nowego kodu *****
    set(fig, 'Renderer', 'zbuffer');
    data_popup_Callback(handles.data_popup, [], handles)
% koniec nowego kodu *****
    if nargin > 0
        varargout{1} = fig;
    end
% ...

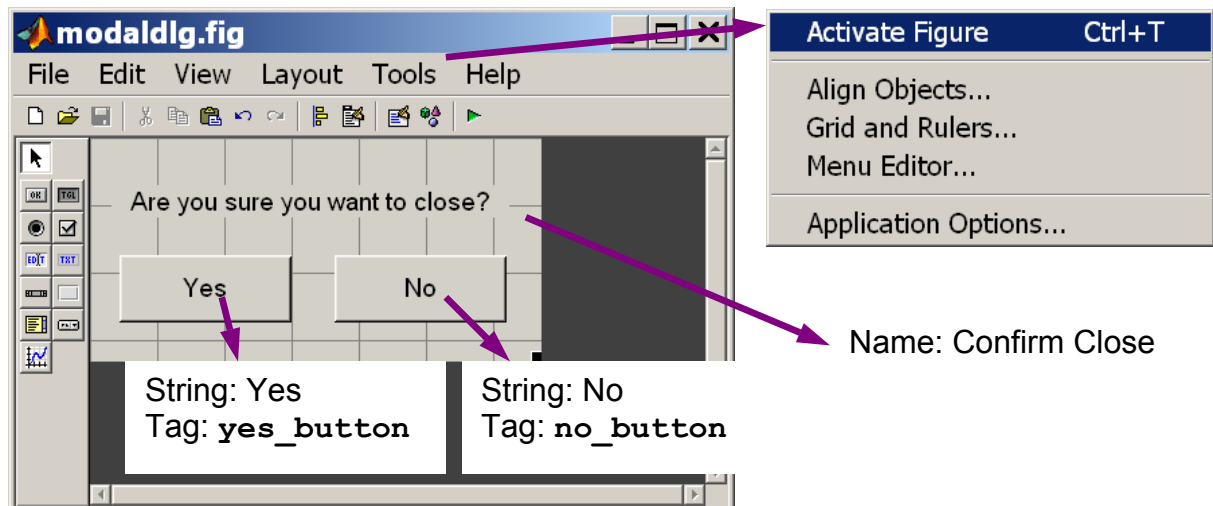
% -----
function varargout = surf_button_Callback(h, eventdata, handles,
varargout)
    z = handles.data; % Load data from handles structure
    surf(z);
% -----
function varargout = mesh_button_Callback(h, eventdata, handles,
varargout)
    z = handles.data; % Load data from handles structure
    mesh(z);
function varargout = contour_button_Callback(h, eventdata,
handles, varargout)
```

```

    z = handles.data; % Load data from handles structure
    contour(z);
% -----
function varargout = data_popup_Callback(h, eventdata, handles,
varargin)
    val = get(h,'Value');
    switch val
        case 1 % User selected peaks
            handles.data = peaks(35);
        case 2 % User selected membrane
            handles.data = membrane;
        case 3 % User selected sinc
            [x,y] = meshgrid(-8:.5:8);
            r = sqrt(x.^2+y.^2) + eps; z = sin(r)./r;
            handles.data = z;
    end
    guidata(h,handles);

```

h) Dodać do projektu okno dialogowe



i) Przeprowadzić edycję kodu źródłowego okna dialogowego

```

function answer = modaldlg(varargin)
if nargin == 0 % LAUNCH GUI
    fig = openfig(mfilename,'reuse');
    set(fig,'WindowStyle','modal'); %modal window
    handles = guihandles(fig);
    guidata(fig, handles);
    uiwait(fig);
    if ~ishandle(fig) %wybrano X zamykający okno
        answer = 'cancel';
    else
        handles = guidata(fig);
        answer = handles.answer;
        delete(fig);
    end
else ...
function varargout = yes_button_Callback(h, eventdata, handles,
varargin)

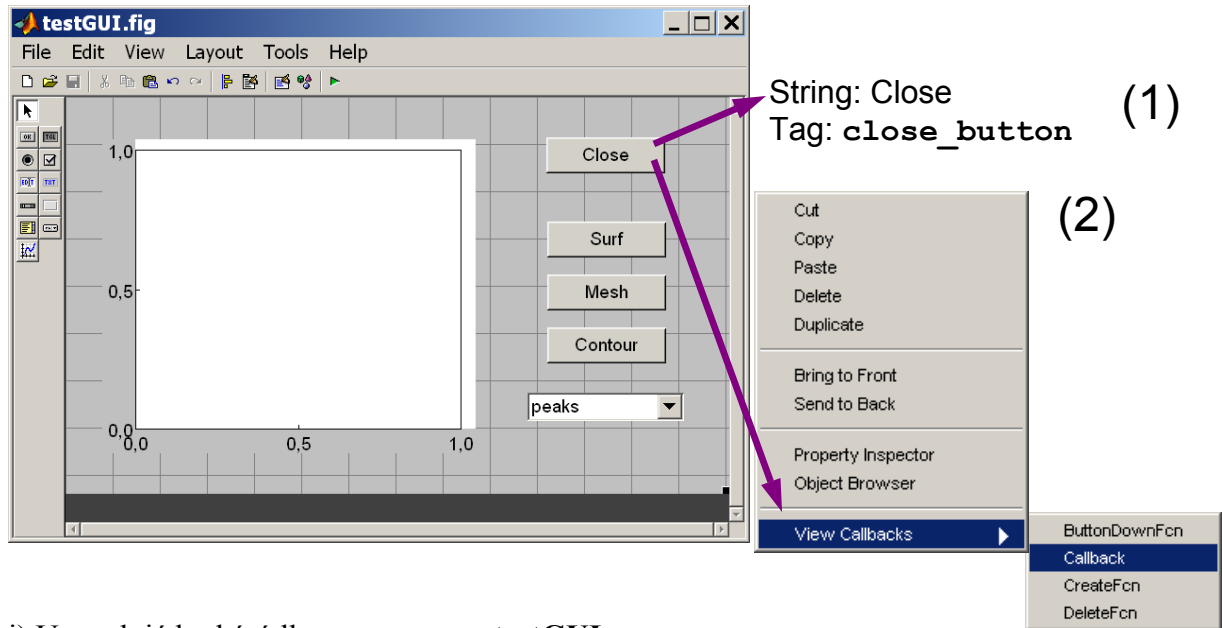
```

```

handles.answer = 'yes';
guidata(h, handles);
uiresume(handles.figure1);
% -----
function varargout = no_button_Callback(h, eventdata, handles,
varargin)
handles.answer = 'no';
guidata(h, handles);
uiresume(handles.figure1);

```

j) Dodać do programu **testGUI** przycisk **Close** otwierający okno dialogowe



j) Uzupełnić kod źródłowy programu **testGUI**

```

user_response = modalDlg; % wyświetlenie okna dialogowego
switch user_response
    case {'no', 'cancel'}
        return
    case 'yes'
        delete(handles.figure1)
end

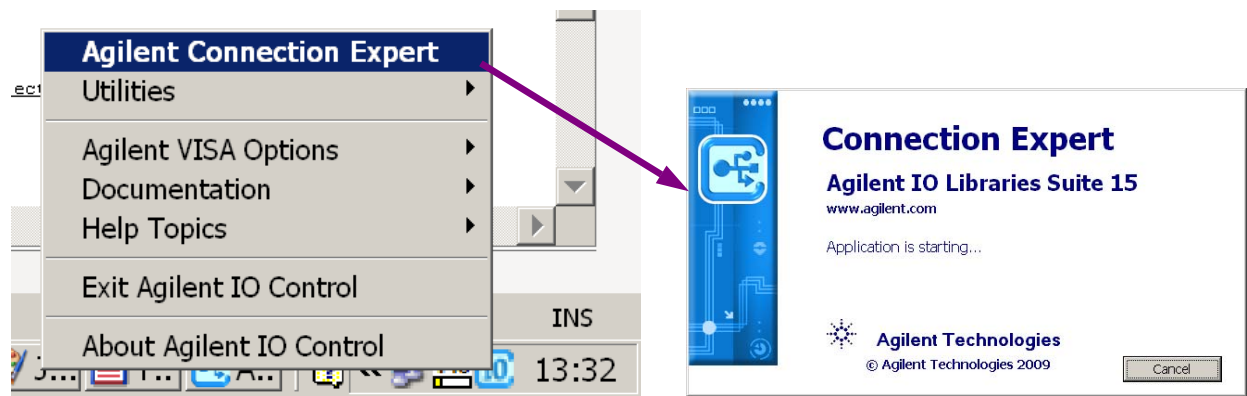
```

k) Zprezentować prowadzącemu efekty pracy.

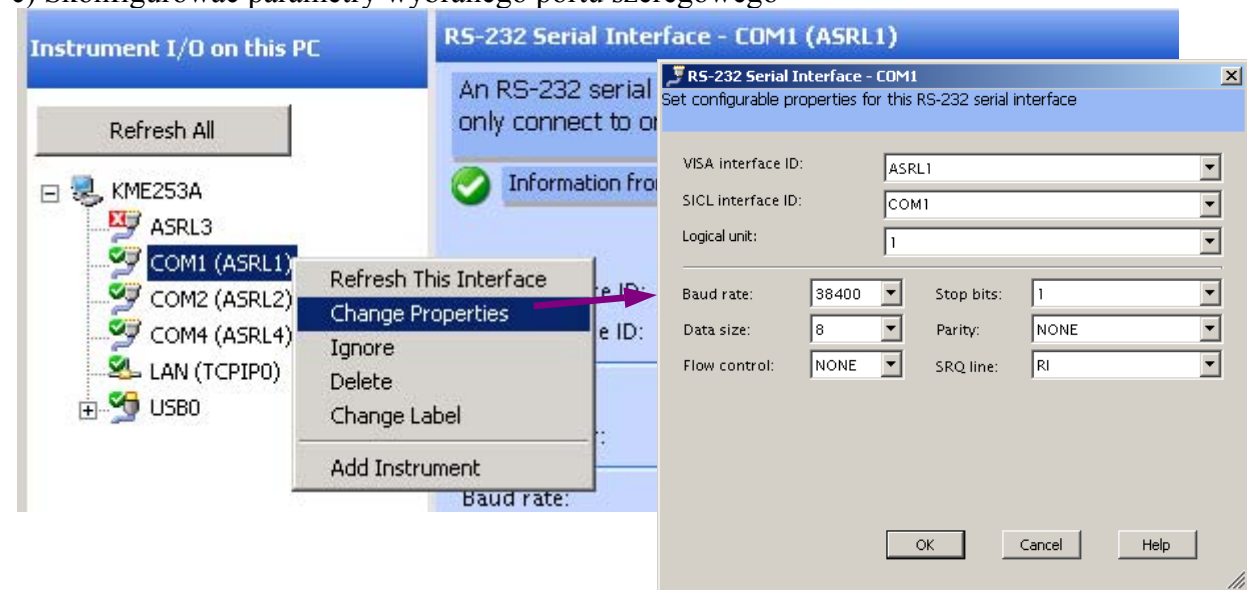
3.3. Sterowanie oscyloskopem RIGOL DS1052E za pomocą komend języka SCPI

a) Ustawić prędkość transmisji przez interfejs szeregowy na 38400
(menu **UTILITY/IO Settings** w oscyloskopie)

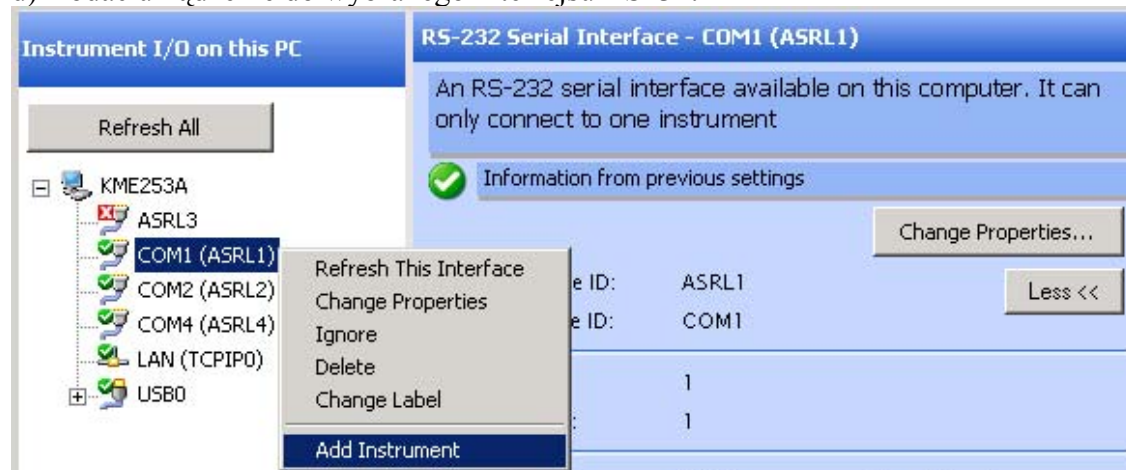
b) Uruchomić program Agilent Connection Expert (ikona **IO** na pasku zadań).



c) Skonfigurować parametry wybranego portu szeregowego



d) Dodać urządzenie do wybranego interfejsu RS232.



e) Wybrać polecenie **Send Commands To This Instrument** i przetestować działanie wybranych komend i zapytań SCPI oscyloskopu ($x = 1, 2$):

***IDN?, :TIM:SCAL, :CHANx:DISP, :CHANx:COUP, :CHANx:SCAL, :CHANx:INV, :CHANx:OFFS, :WAV:POIN:MODE, :WAV:DATA? CHANx**

Znaleźć w dokumentacji znaczenie poszczególnych komend i zapytań.

Zamknąć program **Agilent Connection Expert** i **Agilent Interactive IO**.

3.3. Sterowanie oscyloskopem za pomocą funkcji biblioteki *Instrument Control Toolbox*

- a) Uruchomić MATLABa.
- b) Zapoznać się z funkcją **serial** (**help serial**).
- c) Utworzyć obiekt portu szeregowego **s** i ustawić jego parametry (prędkość liczbą, bitów danych, liczba bitów stopu). Ponadto ustawić właściwość **Timeout** na 1 s i wielkość bufora wejściowego na 100 kB. Wykorzystać funkcję **set**.
- d) Otworzyć dostęp do portu za pomocą funkcji **fopen**.
- e) Sprawdzić działanie wybranych komend SCPI oscyloskopu.
Wykorzystać w tym celu funkcje **fprintf**, **fscanf** i **query**.
- f) Dołączyć dowolny sygnał z generatora do oscyloskopu
- g) Zrejestrować oscylogram i przesłać do komputera za pomocą RS-232:
 - za pomocą odpowiedniej komndy SCPI ustawić liczbę zbieranych próbek na 600
 - zarejestrować oscylogram korzystając z zapytania **:WAV:DATA? CHANx**
 - sprawdzić ile danych jest w buforze wejściowym
 - odczytać dane funkcją **sscanf** i zobaczyć jaki jest ich format.
 - powtórzyć rejestrację, ale tym razem zastosować funkcję **binblockread**. Określić typ danych jako **int8**.
 - Wykreślić odczytane dane za pomocą funkcji **plot**.
- h) Zamknąć dostęp do portu za pomocą funkcji **fclose**.

Uwaga! W razie problemów z komunikacją można wykorzystać funkcje „**instrreset**”, „**close**” lub „**flushinput**”.

3.4. Sterowanie oscyloskopem z wykorzystaniem interfejsu GUI w MATLABie.

- a) Utworzyć nowy panel przyrządu wirtualnego. Umieścić na panelu następujące komponenty:

Axes, Push Button, Listbox, Popup Menu, Checkbox.

Zadaniem kontrolki na panelu powinno być:

- 1) Axes – wyświetlanie oscylogramu;
- 2) Push Button – odczyt oscylogramu z oscyloskopu;
- 3) Listbox – wartość nastawy podstawy czasu (przynajmniej 3 wartości);
- 4) Popup Menu – wybór kanału oscyloskopu (CH1, CH2).

- b) Skonfigurować kontrolki poprzez zmianę wartości parametrów: **Tag**, **String** i/lub **Name**.

- c) Dodać do projektu funkcję **DeleteFcn** wywoływaną przy zamykaniu programu.

W tym celu w edytorze GUI wybrać okno tworzonego programu.

Następnie z menu kontekstowego wybrać **Property Inspector**.

Znaleźć opcję **DeleteFcn** i wpisać **<automatic>**.

W ten sposób funkcja zostanie wygenerowana automatycznie przy aktywacji panelu.

- d) Dokonać aktywacji panelu w celu utworzenia plików *.fig i *.m.

- e) Dodać kod wywoływany przy pierwszym uruchomieniu programu:

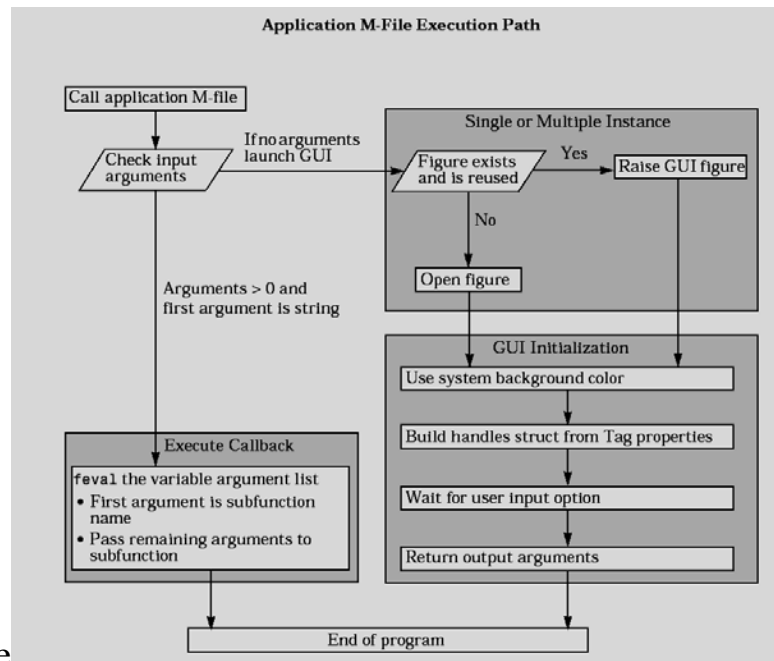
```
set(handles.axes,'XLim',[1 600],'YLim',[-128 127]);
instrreset,close
s = serial('COM1');
set(s,'BaudRate', 38400, 'DataBits', 8, 'StopBits', 1);
set(s,'InputBufferSize', 5000, 'Timeout', 0.5);
fopen(s);
handles.s = s;
guidata(fig, handles);
```

Obiekt portu szeregowego zapisany jest w strukturze **handles** (**handles.s**). Należy go wykorzystywać w funkcjach **callback**.

- f) Funkcja **callback DeleteFcn** powinna zamykać dostęp do portu: **fclose(handles.s)**;
- g) Pozostałe funkcje **callback** uzupełnić samodzielnie.
- h) Zademonstrować działanie programu.

Dodatek

Mechanizm callback w



MATLABie

Obsługa kontrolek na panelu przy pomocy funkcji callback

Ogólna postać funkcji callback dla obiektu, którego pole **Tag** ma wartość `objTagfunc varargout = objTag_Callback(h,eventdata,handles,varargin)`

Parametry funkcji:

- h** – uchwyt do obiektu, który wywołał funkcję
- eventdata** – parametr zarezerwowany
- handles** – struktura zawierająca uchwyty do obiektów na panelu
- varargin** – lista parametrów przekazanych do funkcji callback
- varargout** – wartości zwracane przez funkcję callback

Mechanizm dodawania funkcji callback do kodu źródłowego

Domyślnie, pole **Callback** każdego obiektu na panelu (kontrolka, pole menu) ma nadaną wartość `<automatic>`. Podczas aktywacji panelu, dla wszystkich obiektów, które mają wpisaną wartość `<automatic>` w polu **Callback**, generowana jest funkcja callback. Nazwa tej funkcji bazuje na wartości wpisanej w polu **Tag**.

Po aktywacji panelu zawartość pola **Callback** ulega zmianie.

Przykład:

Nazwa panelu „test”, `Tag = start`, `Callback = <automatic>` =>
`=> Callback = test('start_Callback',gcbo,[],guidata(gcbo))`

Jeśli wartość pola **Callback** jest inna niż `<automatic>`, wówczas funkcja callback nie będzie wygenerowana podczas aktywacji panelu.

Można ją jednak dodać korzystając z menu kontekstowego **View Callbacks/Callback**.

Funkcje wykorzystywane przy programowaniu GUI (1/2)

- `object=gcbo` – pobranie uchwytu do kontrolki, która wygenerowała funkcję callback
 (Get handle to current callback object)
- `[object, figure] = gcbo` – pobranie uchwytu do kontrolki i panelu

figure=gcbf – pobranie uchwytu do panelu, na którym kontrolka wygenerowała funkcję callback (**Get handle to current callback figure**)
handles = guidata(gcbo) – pobranie struktury **handles** zawierającej uchwyt do wszystkich kontrolek na panelu
guidata(gcbo, handles) – zapisanie struktury **handles** (Wywołanie wewnątrz funkcji callback)
guidata(fig, handles) – zapisanie struktury **handles** (Wywołanie poza funkcją callback, **fig** – uchwyt do panelu)

Funkcje wykorzystywane przy programowaniu GUI (2/2)

set(h, 'PropertyName', PropertyValue)
 – ustawienie wartości **PropertyValue** dla atrybutu **PropertyName**.
PropertyValue = get(h, 'PropertyName')
 – pobranie wartości atrybutu **PropertyName**.
feval(f,x1,...,xn) – wywołanie funkcji **f** z parametrami **x1, ..., xn**
[y1,...,yn]= feval(f,x1,...,xn)
 – wywołanie funkcji **f**, która zwraca parametry **y1, ..., yn**
uiwait(fig) – oczekiwanie na zamknięcie okna **fig** lub wywołanie funkcji **uiresume(fig)**

Dostęp do kontrolek na panelu w funkcji callback

Uchwyt do kontrolek na panelu przekazywane są do funkcji **callback** jako pola struktury **handles**. Nazwy pól są takie same jak wartości pól **Tag** przypisanych kontrolkom. Przykładowo aby pobrać napis kontrolki **edit1** (innej niż kontrolka, która wywołała aktualnie obsługiwane zdarzenie), należy wpisać następujący kod

```
napis = get(handles.edit1, 'String');
```

Z kolei aby pobrać wartość kontrolki, która wywołała aktualnie obsługiwane zdarzenie, należy skorzystać z uchwytu **h**, przekazywanego jako pierwszy parametr funkcji callback, np.

```
function varargout =  
togglebutton1_Callback(h,eventdata,handles,varargin)  
    button_state = get(h, 'Value');
```

Obsługa kontrolek:

Przycisk (Push Button)

```
function varargout =  
pushbutton1_Callback(h,eventdata,handles,varargin)  
    % kod funkcji  
    % ...
```

Przełącznik (Toggle Button)

```
function varargout =  
togglebutton1_Callback(h,eventdata,handles,varargin)  
    button_state = get(h, 'Value');  
    if button_state == get(h, 'Max') % toggle button is pressed
```

```

    % ...
elseif button_state == get(h,'Min')
    % toggle button is not pressed
    % ...
end

```

Przełączniki typu Radio Button

```

function varargout = rb1_Callback(h,eventdata,handles,varargin)
    h_rb = [handles.rb2,handles.rb3];
    set(h_rb,'Value','Min');
    % ...
function varargout = rb2_Callback(h,eventdata,handles,varargin)
    h_rb = [handles.rb1,handles.rb3];
    set(h_rb,'Value','Min');
    % ...
function varargout = rb3_Callback(h,eventdata,handles,varargin)
    h_rb = [handles.rb1,handles.rb2];
    set(h_rb,'Value','Min');
    % ...

```

Pole Edit Box

```

function varargout = ebox_Callback(h,eventdata,handles,varargin)
    napis = get(h,'String');
    user_entry = str2double(napis);
    if isnan(user_entry)
        errordlg('You must enter a numeric value','Bad
Input','modal')
    end

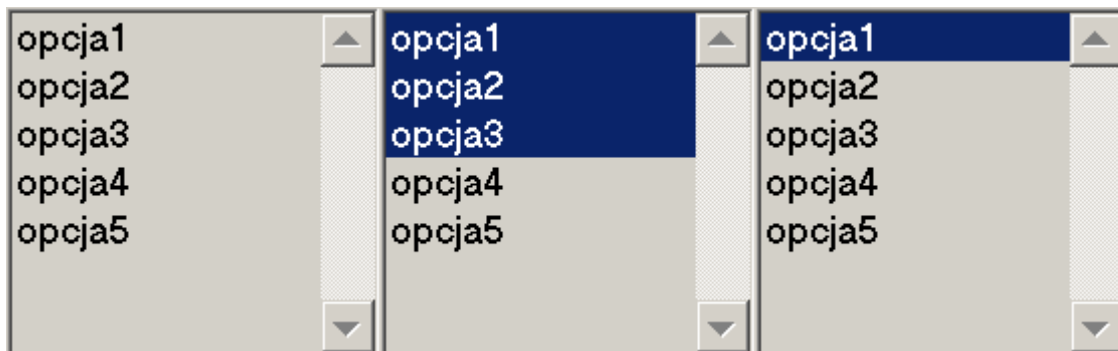
```

Kontrolka List Box

```

function varargout = lbox_Callback(h,eventdata,handles,varargin)
    index = get(h,'Value');
    % index = [] - nie wybrano żadnego elementu
    % index = [1] - wybrano jeden element
    % index = [1 2 3] - wybrano kilka elementów
% Jeśli Max - Min > 1 to kontrolka umożliwia wybór wielokrotny

```



Kontrolka Popup Menu

```

function varargout = pmenu_Callback(h,eventdata,handles,varargin)
    val = get(h,'Value');
    switch val

```

```
case 1
    % The user selected the first item
    % ...
case 2
    % The user selected the second item
    % ...
otherwise
    % ...
end
```

